

Fraud Detection Pipeline Using Supervised Learning

Data Science Project 2 — DecodeLabs Industrial Training Kit
Handling Imbalanced Data with SMOTE, Logistic Regression and Random Forest

Abstract

This project builds a complete supervised learning pipeline to detect fraudulent transactions in an e-commerce order dataset of 1,200 records. Because the dataset contained no fraud labels, a rule-based fraud flag was engineered from realistic risk signals, producing a highly imbalanced target (3.5% fraud). The class imbalance was handled with SMOTE, applied strictly after the train/test split to prevent data leakage. Two classifiers, Logistic Regression and Random Forest, were trained and tuned with grid search cross-validation. Following the project requirements, accuracy was discarded as an evaluation metric in favour of precision, recall and ROC-AUC. Logistic Regression achieved perfect recall (1.000) with ROC-AUC of 0.997, while Random Forest achieved perfect precision (1.000) with ROC-AUC of 0.982, illustrating the fundamental precision–recall trade-off in fraud detection.

1. Introduction and Objective

Fraud detection is one of the most common real-world applications of classification algorithms and also one of the hardest, because fraud is rare. A model that simply predicts “not fraud” for every order would be right about 96% of the time on this dataset while catching zero fraud. The objective of this project is therefore not just to build a classifier, but to build one that is evaluated correctly: using precision, recall and ROC-AUC rather than the misleading accuracy metric.

The specific requirements were to: (1) handle a highly imbalanced dataset using SMOTE, (2) train multiple algorithms (Logistic Regression and Random Forest) using Scikit-Learn, (3) tune hyperparameters and (4) evaluate strictly with precision, recall and ROC-AUC.

2. Dataset Description

The dataset contains 1,200 e-commerce orders with 14 columns: order and customer identifiers, order date, product, quantity, unit price, total price, shipping address, payment method, order status, tracking number, items in cart, coupon code and referral source.

Initial exploration showed the data was largely clean. The only missing values were in CouponCode (309 rows), which simply indicates that no coupon was used, so these were filled with the category “NONE”. A consistency check confirmed that TotalPrice equals Quantity × UnitPrice in every row, and no duplicate records were found.

3. Methodology

3.1 Creating the fraud label

The dataset does not include a fraud column, so a target variable had to be engineered. Following patterns documented in real fraud analysis, an order was flagged as fraudulent when all three of the following conditions held:

- Risky payment method - payment made by Gift Card or Online payment (harder to trace, commonly abused in fraud)
- Suspicious outcome - the order was Cancelled or Returned
- Unusually large order - quantity of 4 or more and a total price above 1,500.

This produced 42 fraudulent orders out of 1,200 (a 3.5% fraud rate), creating the highly imbalanced classification problem the project is designed around. This assumption is stated openly: in a production environment these labels would come from confirmed fraud investigations.

3.2 Feature preparation

Identifier columns (OrderID, CustomerID, ShippingAddress, TrackingNumber) were dropped because they are unique per order and carry no predictive signal. Month and day-of-week features were extracted from the order date. The remaining categorical variables (Product, PaymentMethod, OrderStatus, CouponCode, ReferralSource) were one-hot encoded, yielding 32 features in total.

3.3 Train/test split and scaling

The data was split 80/20 with stratification, so both sets preserve the 3.5% fraud rate (34 fraud cases in training, 8 in testing). Features were standardised with StandardScaler, fitted on the training set only and then applied to the test set, avoiding data leakage.

3.4 Handling imbalance with SMOTE

SMOTE (Synthetic Minority Over-sampling Technique) generates new synthetic minority-class examples by interpolating between existing fraud cases. It was applied only to the training data, after the split - applying SMOTE before splitting would leak synthetic copies of test observations into training and artificially inflate results. After resampling, the training set contained 926 examples of each class.

3.5 Models and hyperparameter tuning

Two classifiers were trained on the balanced training data: Logistic Regression (max_iter = 1000) as an interpretable baseline, and a Random Forest (200 trees, max depth 10). The Random Forest was additionally tuned with GridSearchCV over n_estimators $\in \{100, 200\}$ and max_depth $\in \{5, 10, \text{None}\}$, using 3-fold cross-validation scored by ROC-AUC. The best configuration was 100 trees with max depth 10.

4. Results

Both models were evaluated once, on the untouched 20% test set (240 orders, 8 fraudulent).

Metric	Logistic Regression	Random Forest
Accuracy (reported but discarded)	0.988	0.992

Precision	0.727	1.000
Recall	1.000	0.750
ROC-AUC	0.997	0.982

The confusion matrices show where each model succeeds and fails:

Model	True Positives / False Negatives	False Positives	True Negatives
Logistic Regression	8 caught / 0 missed	3	229
Random Forest	6 caught / 2 missed	0	232

Although both models score around 99% accuracy, this number is deliberately discarded: a trivial model that never predicts fraud would already reach roughly 96.5% accuracy while providing zero value. Precision, recall and ROC-AUC tell the real story.

Feature importance analysis of the Random Forest confirmed that TotalPrice (23%), Quantity (16%), UnitPrice (14%), Gift Card payment (7%) and Cancelled status (5%) were the strongest predictors - consistent with the risk signals used to define the label.

5. Discussion: the precision–recall trade-off

The two models represent opposite ends of the fraud detection trade-off. Logistic Regression caught every fraudulent order (recall = 1.000) at the cost of three false alarms (precision = 0.727). Random Forest raised no false alarms at all (precision = 1.000) but missed two fraudulent orders (recall = 0.750).

Which model is “better” depends on business costs. For financial institutions, a missed fraud case typically costs far more than a false alarm that an analyst can review and dismiss in minutes. Under that assumption, Logistic Regression is the recommended model here: perfect recall, the highest ROC-AUC (0.997), and only three orders flagged unnecessarily out of 240. If analyst review capacity were the binding constraint instead, Random Forest’s zero-false-alarm behaviour would be preferable.

6. Limitations

- The fraud label is rule-based rather than investigator-confirmed, so the models partly re-learn the labelling rules. With real fraud labels, performance would be less clean but more meaningful.
- The dataset is small (1,200 orders, only 8 fraud cases in the test set), so metrics carry high variance; a single missed case moves recall by 12.5 percentage points.
- Only SMOTE was tested; alternatives such as ADASYN, SMOTE-Tomek or class weighting could be compared in future work.

7. Conclusion

This project delivered an end-to-end fraud detection pipeline meeting all stated requirements: an engineered, highly imbalanced target (3.5% fraud), leakage-free SMOTE resampling, two tuned Scikit-Learn classifiers, and strict evaluation by precision, recall and ROC-AUC with accuracy explicitly rejected. Both models separate fraud from legitimate orders almost perfectly (ROC-AUC ≥ 0.98), and the analysis demonstrates the central lesson of imbalanced classification: model choice is ultimately a business decision about which type of error is more expensive.